Reproduce-then-Extend — Building Energy Efficiency (Engineering) (Day 1)

Intro to Machine Learning · Bruce A. Desmarais

July 03, 2026

Published study. Tsanas, A. & Xifara, A. (2012). "Accurate quantitative estimation of energy performance of residential buildings using statistical machine learning tools." *Energy and Buildings* 49:560–567. The paper's baseline is a **linear regression** predicting a building's heating load from 8 geometry inputs (n = 768; UCI id 242). This example illustrates the first regression extension: **polynomial terms**.

1 Background

Tsanas and Xifara study how a building's **energy performance** depends on its **design geometry**, to help architects design more efficient buildings. Using 768 simulated building shapes, they estimate the **heating (and cooling) load** from eight geometry inputs, using a **linear regression** as the interpretable baseline alongside nonlinear machine-learning tools. The **primary conclusion** is that energy load is accurately predictable from geometry – overall height, relative compactness, and glazing are the strongest drivers – and that flexible learners (random forests) substantially outperform the linear model, motivating their use for this strongly nonlinear relationship.

2 Data and codebook

Unit of analysis: a simulated residential **building design** (12 shapes and variants, via Ecotect); n = 768.

Variable	Definition
`Y1`	heating load (kWh per m^2 of floor area) **(outcome)**
`X1`	relative compactness (volume-to-surface ratio, normalized)
`X2`	surface area (m^2)
`X3`	wall area (m^2)
`X4`	roof area (m^2)
`X5`	overall height (m)
`X6`	orientation (2/3/4/5 = N/E/S/W)
`X7`	glazing area (fraction of floor area)
`X8`	glazing-area distribution (0-5 scheme)

3 Descriptive results

```
d <- as.data.frame(read_excel("data/ENB2012_data.xlsx"))   # data ships in the tutorial
↪  folder
d <- d[, colSums(!is.na(d)) > 0]; d <- d[complete.cases(d), ]
preds <- paste0("X", 1:8)
```

```
round(t(sapply(d[c("Y1", preds)],
               function(x) c(mean = mean(x), sd = sd(x), min = min(x), max = max(x)))),
               ↪  2)
```

```
##       mean    sd    min    max
## Y1   22.31 10.09   6.01  43.10
## X1    0.76  0.11   0.62   0.98
## X2  671.71 88.09 514.50 808.50
## X3  318.50 43.63 245.00 416.50
## X4  176.60 45.17 110.25 220.50
## X5    5.25  1.75   3.50   7.00
## X6    3.50  1.12   2.00   5.00
## X7    0.23  0.13   0.00   0.40
## X8    2.81  1.55   0.00   5.00
```

```
par(mfrow = c(1, 2))
par(mar = c(4, 4, 2.5, 1))
hist(d$Y1, breaks = 25, col = "#a6cee3", border = "white", main = "Outcome: Y1 (heating
↪  load)",
     xlab = "heating load")
cors <- sort(sapply(d[preds], function(x) cor(x, d$Y1)))
par(mar = c(4, 4, 2.5, 1))
barplot(cors, horiz = TRUE, las = 1, col = ifelse(cors > 0, "#1f78b4", "#e31a1c"),
        main = "Correlation with Y1", xlab = "correlation")
abline(v = 0, col = "grey50")
```

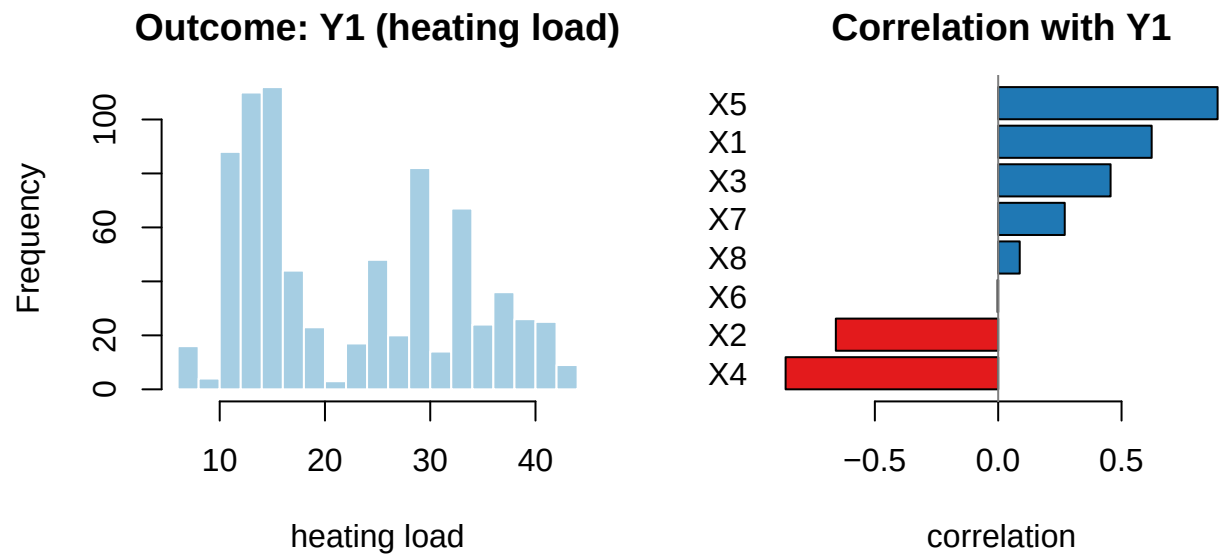

Figure 1: Heating-load distribution (left) and each geometry input's correlation with heating load (right).

4 Reproduce the published regression

```
lin <- lm(Y1 ~ X1 + X2 + X3 + X4 + X5 + X6 + X7 + X8, data = d)
summary(lin)
```

```
##
```

```
## Call:
## lm(formula = Y1 ~ X1 + X2 + X3 + X4 + X5 + X6 + X7 + X8, data = d)
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -9.8965 -1.3196 -0.0252  1.3532  7.7052
##
## Coefficients: (1 not defined because of singularities)
##               Estimate Std. Error t value Pr(>|t|)
## (Intercept)  84.013418  19.033613   4.414 1.16e-05 ***
## X1          -64.773432  10.289448  -6.295 5.19e-10 ***
## X2           -0.087289   0.017075  -5.112 4.04e-07 ***
## X3            0.060813   0.006648   9.148  < 2e-16 ***
## X4                  NA         NA      NA       NA
## X5            4.169954   0.337990  12.338  < 2e-16 ***
## X6           -0.023330   0.094705  -0.246  0.80548
## X7           19.932736   0.813986  24.488  < 2e-16 ***
## X8            0.203777   0.069918   2.915  0.00367 **
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 2.934 on 760 degrees of freedom
## Multiple R-squared:  0.9162, Adjusted R-squared:  0.9154
## F-statistic:  1187 on 7 and 760 DF,  p-value: < 2.2e-16
```

Confirmation against the published study. The linear model attains in-sample $R^2 = 0.916$, matching the ~0.92 reported by Tsanas & Xifara for their linear baseline. Overall height (`X5`) and relative compactness (`X1`) dominate — but the fit leaves clear structure, which is the motivation for going beyond a straight line.

5 Polynomial regression extension

Heating load responds **nonlinearly** to geometry. The classic regression remedy is to add **polynomial terms**. We compare the linear model to polynomials of increasing degree in relative compactness (`X1`), keeping the other inputs linear. We pick the degree by cross-validation on a training set and report final RMSE on a held-out test set.

```r
set.seed(2026)
others <- paste0("X", 2:8)
te <- sample(nrow(d), round(0.30 * nrow(d)))            # hold out a 30% test set up
↪  front
tr <- setdiff(seq_len(nrow(d)), te)

# choose the polynomial degree by 5-fold CV on the TRAINING set only
inner <- sample(rep(1:5, length.out = length(tr)))
cv_rmse <- sapply(1:4, function(deg) {
  r <- numeric(5)
  for (k in 1:5) {
    itr <- tr[inner != k]; ite <- tr[inner == k]
    form <- reformulate(c(sprintf("poly(X1, %d, raw = TRUE)", deg), others), "Y1")
    r[k] <- sqrt(mean((d$Y1[ite] - predict(lm(form, data = d[itr, ]), d[ite, ]))^2))
  }
  mean(r)
})
names(cv_rmse) <- paste0("degree ", 1:4)
best_deg <- which.min(cv_rmse)
```

```r
round(cv_rmse, 3)                                   # CV RMSE per degree (training set)
```

```
## degree 1 degree 2 degree 3 degree 4
##    2.957    2.924    2.836    2.831
```

```r
# refit at the CV-chosen degree on the full training set; score ONCE on the untouched
↪  test set
form_best <- reformulate(c(sprintf("poly(X1, %d, raw = TRUE)", best_deg), others), "Y1")
test_poly <- sqrt(mean((d$Y1[te] - predict(lm(form_best, data = d[tr, ]), d[te, ]))^2))
test_lin  <- sqrt(mean((d$Y1[te] - predict(lm(reformulate(c("X1", others), "Y1"), data =
↪  d[tr, ]), d[te, ]))^2))
cat(sprintf("CV-selected degree: %d.  Held-out test RMSE: polynomial = %.3f vs linear =
↪  %.3f
",
            best_deg, test_poly, test_lin))
```

```
## CV-selected degree: 4.  Held-out test RMSE: polynomial = 2.780 vs linear = 2.950
```

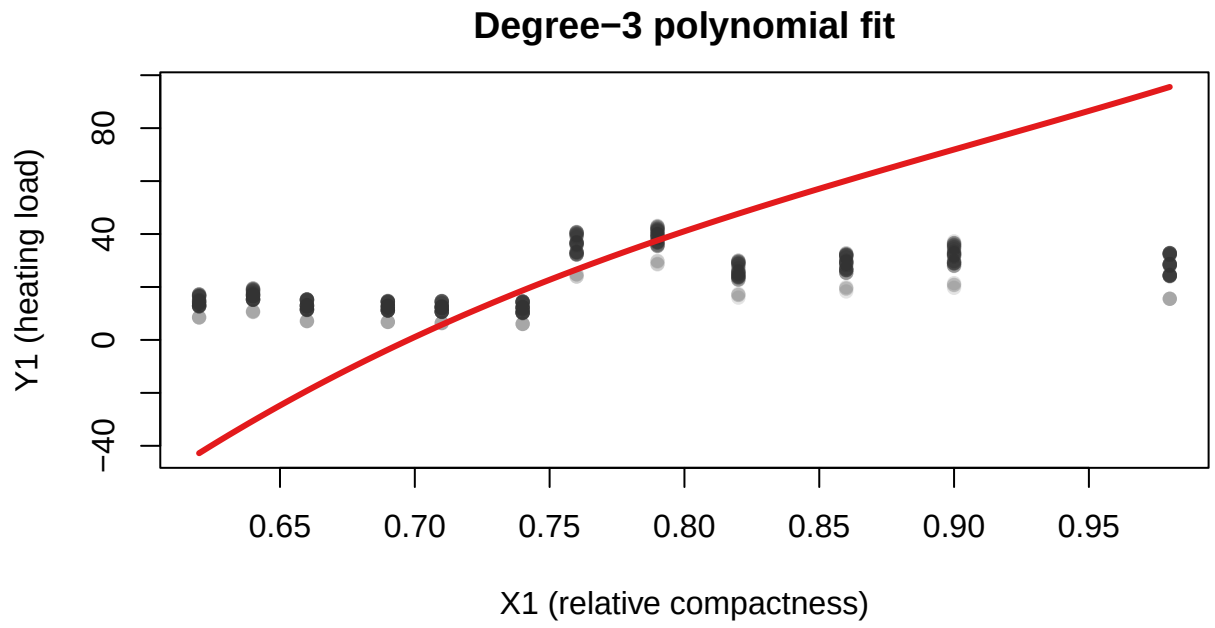

Figure 2: Degree-3 polynomial fit of heating load on relative compactness, holding the other inputs at their means.

On the held-out test set the CV-selected polynomial improves on the straight-line fit; the curve **bends** with compactness rather than running straight, capturing nonlinearity the linear model misses while remaining a fully interpretable regression.

6 Takeaway

This example illustrates **polynomial regression** as the first, most interpretable extension of a linear model: adding low-order polynomial terms lowers cross-validated error by bending the fit to the data's curvature. (Tree-based learners push this further at the cost of a closed-form equation; Tsanas & Xifara report ML reaching ~0.999 R^2 on these data.)

7 Recommended exercises

1. Try polynomial degrees 1-5 for the strongest predictor and use cross-validation to choose the degree.
2. Predict the second response (cooling load) and compare which predictors matter most.
3. Add an interaction between two predictors and test whether cross-validated error improves.